

CONTROLS SYSTEMS

Project 09

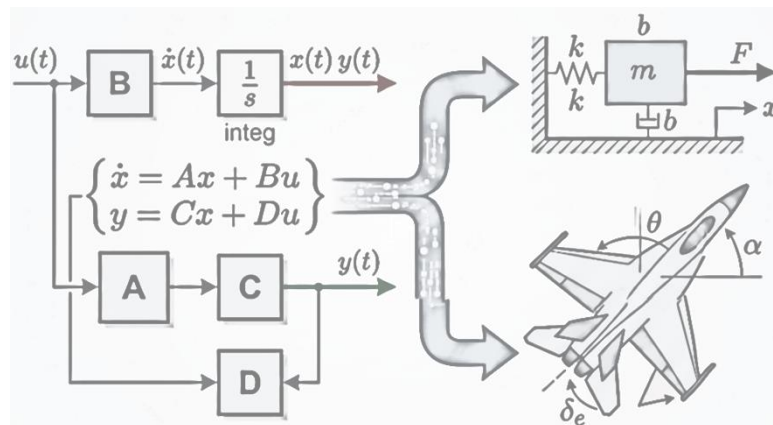
STATE-SPACE MODELING AND ANALYSIS OF MECHANICAL AND AEROSPACE SYSTEMS

Designed, Developed and Documented by

Zohaib Imtiaz

MATLAB R2024b • Control System Toolbox • Aerospace Engineering

August 2026



Copyright © 2026 Zohaib Imtiaz. All Rights Reserved.

This report is the original work of the author and has been prepared for academic purposes. No part of this document may be reproduced, distributed, or transmitted in any form without proper acknowledgment of the author, except where permitted for educational or research purposes.

Abstract

State-space representation provides a systematic method for modeling and analyzing dynamic systems using internal state variables. Unlike the classical transfer-function approach, state-space modeling describes the internal dynamics of a system through a set of first-order differential equations and provides a foundation for modern control techniques.

This project investigates state-space modeling in two stages. First, a mass–spring–damper system is represented in state-space form and analyzed through state responses, eigenvalues, parameter variation, and controllability. The methodology is then extended to a third-order aircraft pitch-control system. The aircraft model is analyzed through its state-space matrices, eigenvalues, controllability, observability, and comparison with its transfer-function representation. The agreement between transfer-function and state-space responses verifies the correctness of the developed models and establishes the foundation for future modern control design.

Table of Contents

Abstract	iii
List of Tables	vii
Table of Figures	viii
1 Objectives	1
2 Introduction.....	2
3 Background	3
3.1 State-Space Representation.....	3
3.2 Why State Space Representation Is Important?.....	4
4 Mass–Spring–Damper State-Space Model	5
4.1 Governing Equation	5
4.2 Definition of States	5
4.3 Transfer-Function Representation	6
5 MATLAB Implementation — Mass–Spring–Damper	6
5.1 State-space construction.....	6
5.2 Pole and eigenvalue calculation.....	7
5.3 Transfer function.....	7
5.4 State-Space and Transfer-Function Verification	8
5.4.1 Step response	8
5.4.2 Impulse response.....	9
5.4.3 Observation	9
5.5 Individual State Visualization.....	10
5.5.1 Responses.....	10
5.5.2 Observation	11
5.6 Eigenvalue Analysis.....	11
5.7 Parameter Study Using Eigenvalues	12
5.7.1 Effect of Increasing Mass	12
5.7.1.1 Effect on Eigen Values	12

5.7.1.2	Graphical Representation.....	13
5.7.1.3	Observation	13
5.7.2	Effect of Increasing Damping.....	13
5.7.2.1	Effect on Eigen Values	13
5.7.2.2	Graphical Representation.....	14
5.7.2.3	Observation	14
5.7.3	Effect of Increasing Spring Stiffness	14
5.7.3.1	Effect on Eigen Values	15
5.7.3.3	Observation	15
5.8	Controllability	16
5.8.1	MATLAB Output.....	16
5.8.2	Uncontrollable Test Case.....	17
6	Transition to Aircraft Pitch Control.....	18
6.1	Aircraft Pitch State-Space Model	18
6.2	MATLAB Verification	19
6.2.1	Why Are There Three States?.....	19
6.3	Aircraft Poles and Eigenvalues	20
6.4	Aircraft Controllability	20
6.4.1	MATLAB Output.....	20
6.4.2	Significance of the Result	21
6.5	Aircraft Observability	21
6.5.1	MATLAB Output.....	21
6.5.2	Significance of the Result	22
6.6	Transfer Function vs State-Space Verification — Aircraft	22
6.6.1	Step Response	22
7	Results and Discussion	24
7.1	Mass–Spring–Damper.....	24

7.2	Eigenvalue Study	24
7.3	Controllability	24
7.4	Aircraft State-Space Model.....	25
8	Aerospace Applications	26
9	Conclusion	27
10	Future Work	28
11	Appendix.....	29
11.1	MATLAB Code	29

List of Tables

<i>Table 1 Eigenvalue movement with increasing mass</i>	<i>12</i>
<i>Table 2 Eigenvalue movement with increasing damping</i>	<i>13</i>
<i>Table 3 Eigenvalue movement with increasing stiffness.....</i>	<i>15</i>

Table of Figures

Figure 1 Eigenvalue movement with increasing mass.....	8
Figure 2 Eigenvalue movement with increasing damping.....	9
Figure 3 Position (x_1) vs. Velocity (x_2)	10
Figure 4 Eigenvalue movement with increasing mass.....	13
Figure 5 Eigenvalue movement with increasing damping.....	14
Figure 6 Eigenvalue movement with increasing stiffness	15
Figure 7 Aircraft transfer-function vs state-space step response	22

1 Objectives

The specific objectives are:

- To understand the concept of state-space representation.
- To identify suitable state variables for a dynamic system.
- To derive the state-space model of a mass–spring–damper system.
- To verify the state-space model against its transfer-function representation.
- To visualize individual state responses.
- To investigate the effect of mass, damping, and spring stiffness on state-space eigenvalues.
- To understand the relationship between system poles and eigenvalues.
- To analyze controllability.
- To investigate observability for the aircraft pitch system.
- To develop a state-space representation of the aircraft pitch plant.
- To verify the aircraft state-space model against its original transfer function.
- To establish a foundation for future modern control techniques such as pole placement and optimal control.

2 Introduction

In previous control-system studies, dynamic systems were primarily represented using transfer functions. A transfer function provides a convenient relationship between an input and an output and is particularly useful for classical techniques such as root-locus, Bode analysis, Nyquist analysis, and compensator design.

However, a transfer function does not explicitly describe the internal variables responsible for the system's dynamic behavior. This becomes increasingly important for higher-order systems and aerospace applications, where several internal variables may influence the system simultaneously.

State-space representation addresses this limitation by expressing a dynamic system using a set of first-order differential equations:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t)$$

and an output equation:

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t)$$

where the vector contains the system states.

In this project, state-space modeling is first introduced using a mass–spring–damper system because its physical states can be understood easily. The same framework is then applied to the aircraft pitch-control plant studied in the previous projects.

This creates progression from a simple mechanical system to an aerospace control system and demonstrates how the same mathematical framework can be used for increasingly complex dynamic systems.

3 Background

3.1 State-Space Representation

A general continuous-time state-space model is written as:

$$\dot{x}(t) = Ax(t) + Bu(t)$$

Output Equation:

$$y(t) = Cx(t) + Du(t)$$

where:

Variable Definitions

- x : State vector
- $\dot{x}$: Time derivative of the state vector ($\frac{dx}{dt}$)
- u : Input vector (control inputs)
- y : Output vector (measured quantities)
- A : System / State matrix
- B : Input matrix
- C : Output matrix
- D : Direct-feedthrough matrix

The state vector contains the minimum number of variables required to describe the instantaneous condition of the dynamic system.

For an n-th- order system, a minimal realization generally requires independent states.

3.2 Why State Space Representation Is Important?

A transfer function primarily describes the input-output behavior of a system:

$$u \rightarrow G(s) \rightarrow y$$

State-space representation provides information about internal dynamics:

$$u \rightarrow \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \rightarrow y$$

This becomes particularly useful when:

- the system has multiple states,
- there are multiple inputs or outputs,
- state feedback is required,
- controllability needs to be examined,
- observability needs to be examined,
- modern control techniques are applied.

State-space modeling therefore provides the mathematical foundation for techniques such as pole placement, LQR, and state estimation.

4 Mass–Spring–Damper State-Space Model

4.1 Governing Equation

The mass–spring–damper system is governed by:

$$m\ddot{x} + b\dot{x} + kx = F$$

where:

- m = mass
- b = damping coefficient
- k = spring stiffness
- x = displacement
- F = applied force

Rearranging:

$$\ddot{x} = -\frac{k}{m}x - \frac{b}{m}\dot{x} + \frac{1}{m}F$$

4.2 Definition of States

Two states are selected:

$$x_1 = x$$

$$x_2 = \dot{x}$$

Therefore:

$$\dot{x}_1 = x_2$$

and:

$$\dot{x}_2 = -\frac{k}{m}x_1 - \frac{b}{m}x_2 + \frac{1}{m}F$$

Thus:

For the selected values ($m = b = k = 1$), we obtain:

$$A = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad C = [1 \quad 0], \quad D = [0]$$

4.3 Transfer-Function Representation

The mass–spring–damper transfer function is:

$$G(s) = \frac{X(s)}{F(s)} = \frac{1}{ms^2 + bs + k}$$

For $m = b = k = 1$, the transfer function becomes:

$$G(s) = \frac{1}{s^2 + s + 1}$$

The state-space model is converted back into a transfer function using MATLAB to verify that both representations describe the same dynamic system.

5 MATLAB Implementation — Mass–Spring–Damper

Following results are the outputs of MATLAB, which were performed for the verification

5.1 State-space construction

```
A = [0 1; -k/m -b/m];  
B = [0 ; 1];  
C = [1 0];  
D = [0];  
sys_ss = ss(A,B,C,D)
```

Output:

```
sys_ss =
```

```
A =  
      x1    x2  
x1      0     1  
x2     -1    -1
```

```
B =  
      u1  
x1      0
```

```

    x2    1

C =
      x1    x2
y1    1    0

D =
      u1
y1    0

```

5.2 Pole and eigenvalue calculation

```

poles = pole(sys_ss)
eigenValues = eig(A)

```

Output:

```

poles =

    -0.5000 + 0.8660i
    -0.5000 - 0.8660i

eigenValues =

    -0.5000 + 0.8660i
    -0.5000 - 0.8660i

```

5.3 Transfer function

```

num = 1 ;
den = [m b k];
sys_tf = tf(num,den);

```

Output:

```

sys_tf =

      1
-----
s^2 + s + 1

```

The state-space model and transfer-function model are then compared using step and impulse responses.

5.4 State-Space and Transfer-Function Verification

Two verification tests are performed:

5.4.1 Step response

The step responses of the state-space model and the transfer-function model are plotted together.

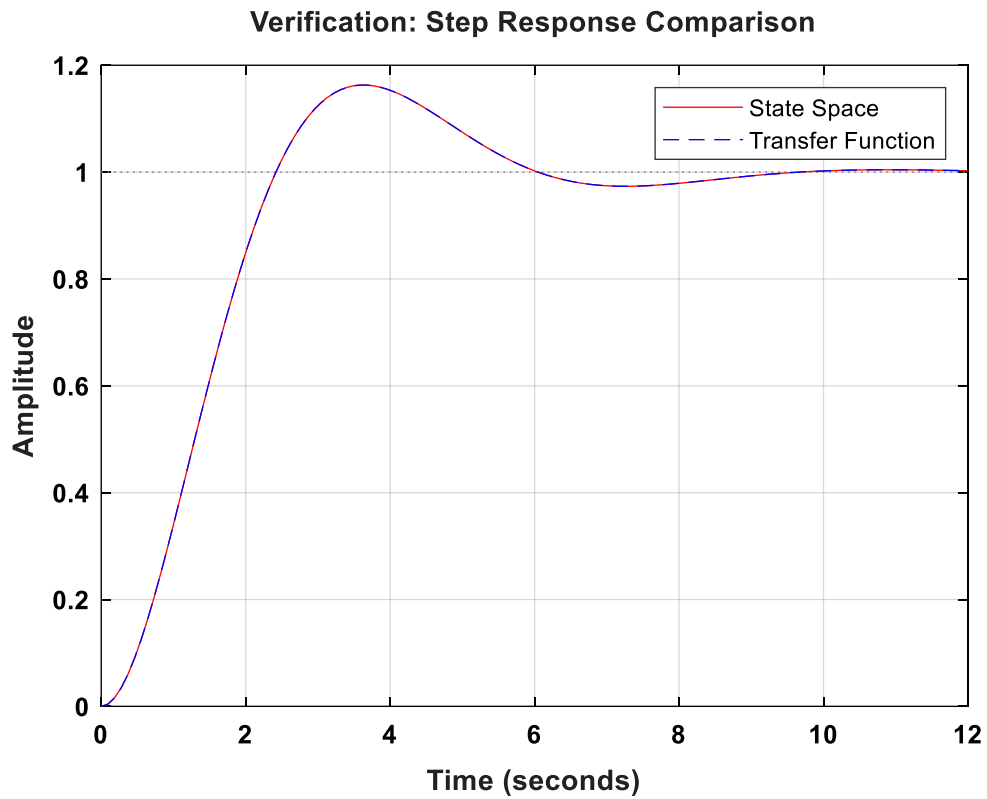


Figure 1 Eigenvalue movement with increasing mass

5.4.2 Impulse response

The impulse responses of both models are also compared.

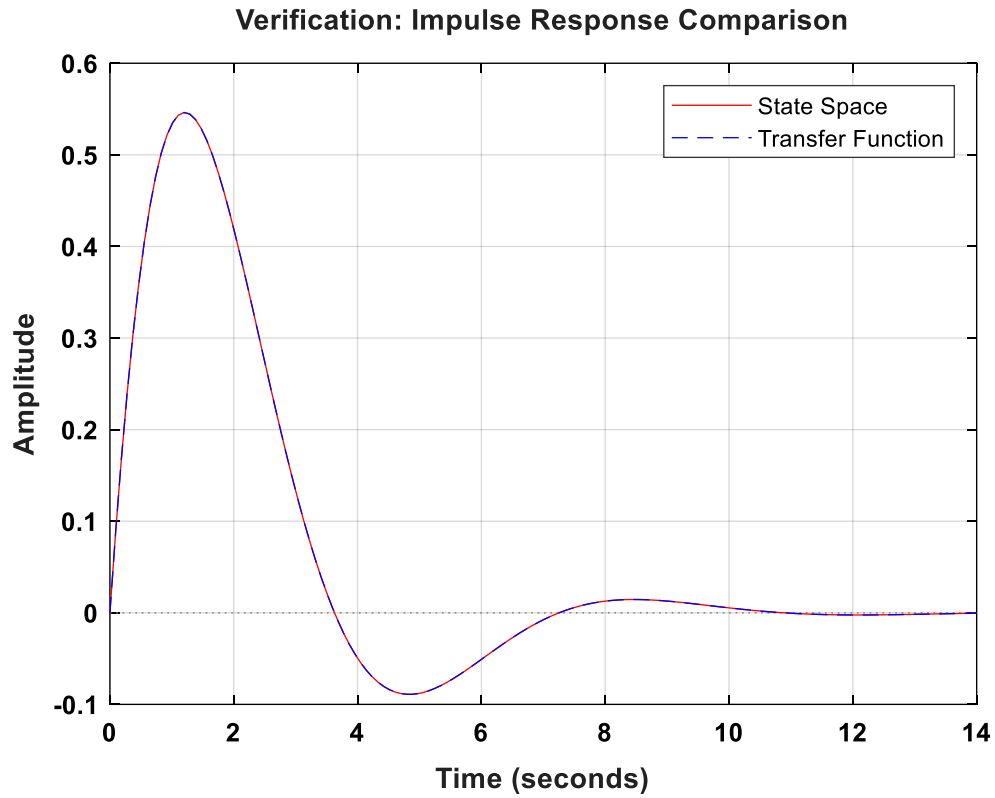


Figure 2 Eigenvalue movement with increasing damping

5.4.3 Observation

The responses overlap, this indicates that the state-space representation correctly represents the same input-output dynamics as the transfer function.

This is important because a state-space representation is not intended to create a different physical system, it is simply another mathematical representation of the same dynamic behavior.

5.5 Individual State Visualization

One of the major advantages of state-space modeling is that individual states can be selected as outputs.

For the mass–spring–damper system:

$$x_1 = x$$

represents position, while:

$$x_2 = \dot{x}$$

represents velocity.

The output matrices are therefore changed to:

$$C_{\text{position}} = [1 \quad 0]$$

and:

$$C_{\text{velocity}} = [0 \quad 1]$$

5.5.1 Responses

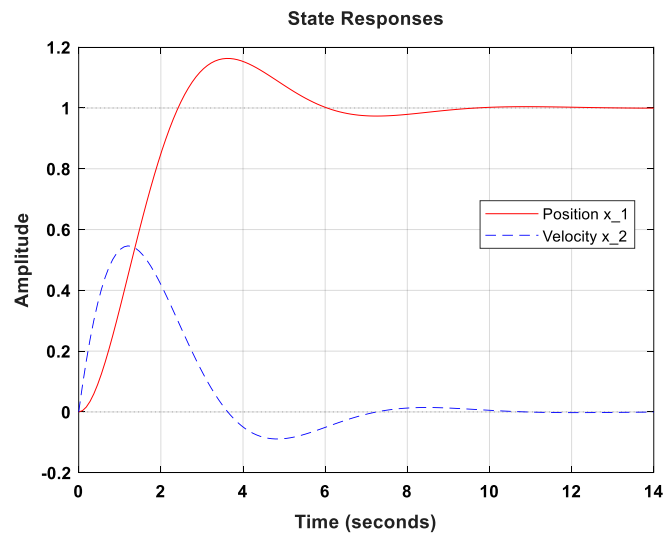


Figure 3 Position (x_1) vs. Velocity (x_2)

5.5.2 Observation

Key Takeaway: The transfer function approach primarily shows the system's external input–output behavior. Here, state-space modeling allows us to inspect the internal state variables individually.

5.6 Eigenvalue Analysis

For a state-space system, the eigenvalues of the system matrix A determine the natural dynamic modes of the system.

For:

$$A = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix}$$

the eigenvalues are:

$$\lambda_{1,2} = -0.5 \pm 0.866j$$

The poles obtained from the state-space model are identical:

$$p_{1,2} = -0.5 \pm 0.866j$$

Therefore:

$$\lambda(A) = \text{poles of the system}$$

for this realization.

5.7 Parameter Study Using Eigenvalues

Unlike Project 01, where the parameter variations were primarily interpreted through transfer-function poles and responses, this project examines the same physical parameters through the state-space system matrix.

The state-space model matrix is:

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{b}{m} \end{bmatrix}$$

Therefore, changing m , b , or k directly changes the elements of A , which consequently changes its eigenvalues.

5.7.1 Effect of Increasing Mass

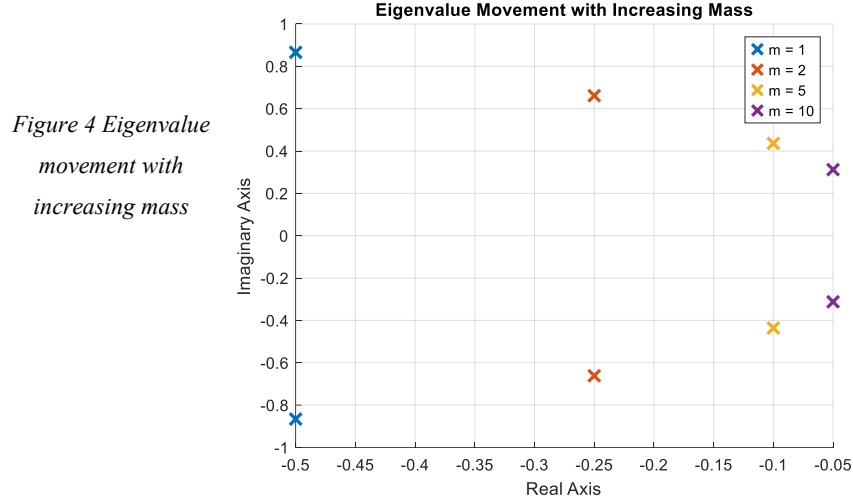
The values $m = 1, 2, 5, 10$ are investigated while $b = k = 1$ remain constant.

5.7.1.1 Effect on Eigen Values

Table 1 Eigenvalue movement with increasing mass

Mass (m)	Eigenvalues ($\lambda_{1,2}$)
1	$-0.5 \pm 0.866j$
2	$-0.25 \pm 0.6614j$
3	$-0.1 \pm 0.4359j$
4	$-0.05 \pm 0.3122j$

5.7.1.2 Graphical Representation



5.7.1.3 Observation

As mass increases, the real part moves toward the imaginary axis and the imaginary component decreases. This indicates that increasing mass changes both the damping-related behavior and the oscillatory characteristics of the system.

The movement of the eigenvalues provides a direct state-space interpretation of how increasing inertia affects overall system dynamics.

5.7.2 Effect of Increasing Damping

The values $b = 1, 2, 5, 10$ are investigated while $m = k = 1$ remain constant.

5.7.2.1 Effect on Eigen Values

Table 2 Eigenvalue movement with increasing damping

Damping (b)	Eigenvalues ($\lambda_{1,2}$)
1	$-0.5 \pm 0.866j$
2	$-1, -1$
3	$-0.2087, -4.7913$
4	$-0.1010, -9.8990$

5.7.2.2 Graphical Representation

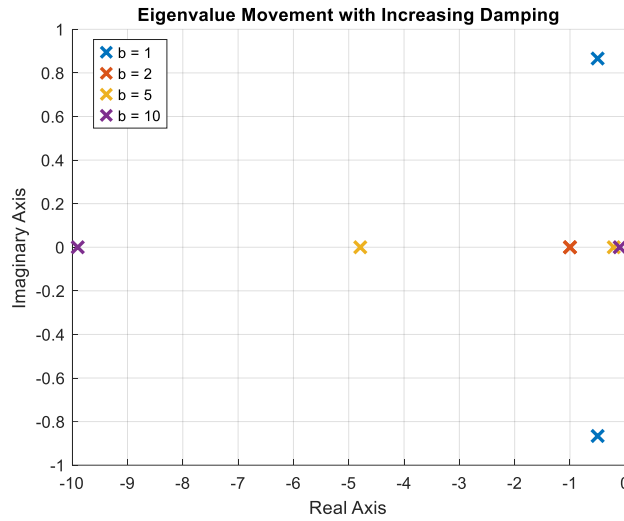


Figure 5 Eigenvalue movement with increasing damping

5.7.2.3 Observation

- At $b = 1$, the eigenvalues are complex conjugates, indicating underdamped (oscillatory) behavior.
- At $b = 2$, the eigenvalues become repeated real poles (critically damped).
- For larger damping values ($b = 5, 10$), the eigenvalues split into two distinct negative real values (overdamped).

Increasing damping progressively suppresses oscillatory behavior and shifts the system toward an overdamped response.

5.7.3 Effect of Increasing Spring Stiffness

The values $k = 1, 2, 5, 10$ are investigated while $m = b = 1$ remain constant.

5.7.3.1 Effect on Eigen Values

Table 3 Eigenvalue movement with increasing stiffness

Stiffness (k)	Eigenvalues ($\lambda_{1,2}$)
1	$-0.5 \pm 0.866j$
2	$-0.5 \pm 1.3229j$
5	$-0.5 \pm 2.1794j$
10	$-0.5 \pm 3.1225j$

5.7.3.2 Graphical Representation

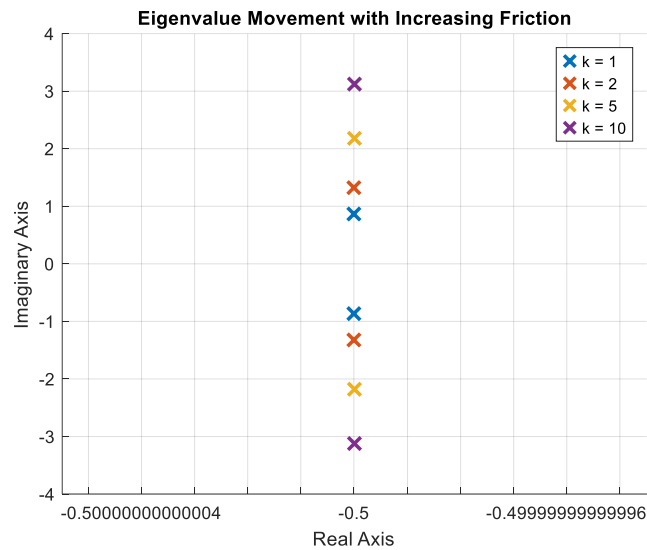


Figure 6 Eigenvalue movement with increasing stiffness

5.7.3.3 Observation

The real part remains strictly constant at -0.5 , while the magnitude of the imaginary component increases.

Increasing spring stiffness directly increases the natural oscillatory frequency of the system without altering the exponential decay rate set by the real component.

5.8 Controllability

A state-space system is controllable if its input can influence all independent states of the system.

The controllability matrix is defined as:

$$\mathcal{C} = [B \quad AB \quad A^2B \quad \dots \quad A^{n-1}B]$$

For the second-order mass–spring–damper system ($n = 2$):

$$\mathcal{C} = [B \quad AB]$$

5.8.1 MATLAB Output

```
A =  
    0    1  
   -1   -1  
B =  
    0  
    1  
n =  
    2  
Co =  
    0    1  
    1   -1  
Rank =  
  
    2  
  
The system is completely controllable
```

Since:

$$\text{rank}(\mathcal{C}) = n$$

Therefore, **the mass–spring–damper system is completely controllable.**

5.8.2 Uncontrollable Test Case

$$B = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This represents a system in which no control input is applied.

The resulting controllability matrix has:

$$\text{rank}(\mathcal{C}) = 0$$

Since $0 < 2$, the system is not controllable.

This test provides a simple verification of what controllability means physically: if the input cannot influence the system, the states cannot be controlled through that input

6 Transition to Aircraft Pitch Control

After establishing the fundamental concepts of state-space representation using the mass–spring–damper system, the same methodology was applied to the aircraft pitch-control plant previously studied using classical control techniques. This provides a direct transition from a simple second-order mechanical system to a third-order aerospace control system.

The aircraft plant transfer function is:

$$G(s) = \frac{-1.282s + 1.282}{s^3 + 1.935s^2 + 0.987s + 0.179}$$

6.1 Aircraft Pitch State-Space Model

The state-space representation obtained in Controllable Canonical Form is:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -0.179 & -0.987 & -1.935 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$
$$C = [1.282 \quad -1.282 \quad 0], \quad D = [0]$$

Therefore, the state-equation dynamic model is:

$$\dot{x} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -0.179 & -0.987 & -1.935 \end{bmatrix} x + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \delta_e$$

and the output equation for the pitch angle θ is:

$$\theta = [1.282 \quad -1.282 \quad 0]x$$

6.2 MATLAB Verification

The same model was performed on MATLAB for verification:

```
----- STATE SPACE REPRESENTATION -----  
  
sys_ss =  
  
A =  
      x1      x2      x3  
x1      0      1      0  
x2      0      0      1  
x3 -0.179 -0.987 -1.935  
  
B =  
      u1  
x1      0  
x2      0  
x3      1  
  
C =  
      x1      x2      x3  
y1  1.282 -1.282      0  
  
D =  
      u1  
y1      0  
  
----- TRANSFER FUNCTION REPRESENTATION --  
  
sys_tf =  
  
      -1.282 s + 1.282  
-----  
s^3 + 1.935 s^2 + 0.987 s + 0.179
```

6.2.1 Why Are There Three States?

The aircraft transfer function has a third-order denominator:

$$s^3 + 1.935s^2 + 0.987s + 0.179$$

Therefore, a minimal state-space realization requires three states:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

In the **controllable canonical realization** used here, these states are mathematical state coordinates generated from the canonical transformation rather than direct physical assignments (such as explicitly setting them to pitch angle θ , pitch rate q , and angle of attack α).

6.3 Aircraft Poles and Eigenvalues

The poles obtained from the aircraft state-space model are:

```
-0.3336 + 0.1730i
-0.3336 - 0.1730i
-1.2679 + 0.0000i
```

The eigenvalues of A are identical:

```
-0.3336 + 0.1730i
-0.3336 - 0.1730i
-1.2679 + 0.0000i
```

Thus:

Poles of the system = Eigenvalues of A

This verifies that the state-space model preserves the natural dynamics of the original aircraft plant.

6.4 Aircraft Controllability

The controllability matrix for a third-order system ($n = 3$) is:

$$\mathcal{C} = [B \quad AB \quad A^2B]$$

6.4.1 MATLAB Output

```
Controllable Matrix:
      0      0      1.0000
      0      1.0000     -1.9350
      1.0000     -1.9350      2.7572

Rank: 3
```

Since the aircraft system has:

$$n = 3$$

we have:

$$\text{rank}(\mathcal{C}) = n$$

Therefore:

The aircraft pitch system is completely controllable.

6.4.2 Significance of the Result

This is a major result for the state-space analysis. It proves mathematically that the elevator input (δ_e) can, through the internal system dynamics, influence all three independent states of the model. This full controllability guarantees that full state feedback controllers (such as pole placement or LQR) can be designed to arbitrarily place all closed-loop poles.

6.5 Aircraft Observability

Observability asks the opposite question of controllability:

Can the internal states of the system be inferred solely from the available output measurements?

The observability matrix for a third-order system ($n = 3$) is:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$$

6.5.1 MATLAB Output

```
Observability Matrix:
    1.2820    -1.2820         0
         0     1.2820    -1.2820
    0.2295     1.2653     3.7627

Rank:
     3
```

Since $n = 3$, we have:

$$\text{rank}(\mathcal{O}) = n$$

Therefore:

The aircraft pitch system is completely observable.

6.5.2 Significance of the Result

Because the system is completely observable, the internal state trajectories are theoretically recoverable purely from measuring the output pitch angle (θ). This property guarantees that an observer or estimator (such as a Luenberger observer or Kalman filter) can be constructed to reconstruct all internal states accurately without needing direct sensor measurements for each individual state.

6.6 Transfer Function vs State-Space Verification — Aircraft

The final verification compares the original transfer function model $G(s)$ with the state-space representation (A, B, C, D) using their step responses.

6.6.1 Step Response

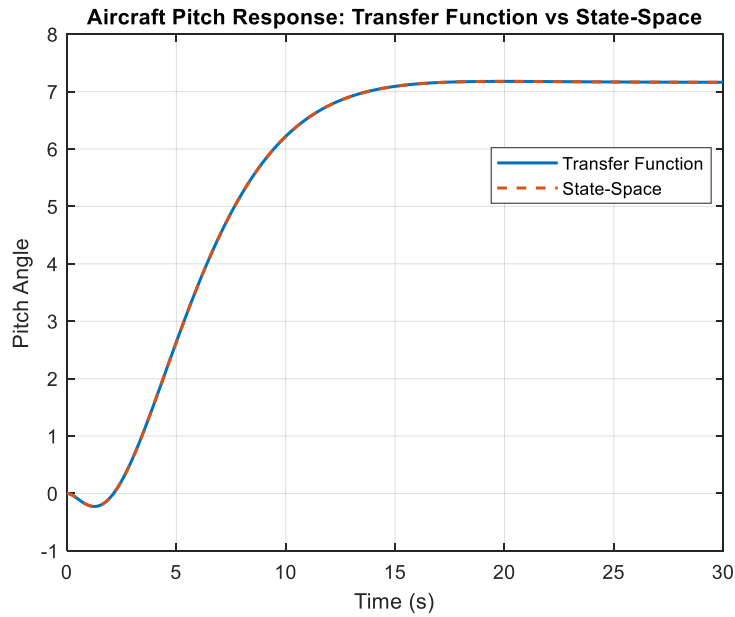


Figure 7 Aircraft transfer-function vs state-space step response

This MATLAB result show that the transfer-function and state-space responses overlap completely.

This demonstrates that the developed state-space representation correctly reproduces the dynamic input-output behavior of the original aircraft transfer function.

Therefore:

$$\textit{Aircraft Transfer Function} \equiv \textit{Aircraft State-Space Model}$$

in terms of the modeled input-output behavior.

7 Results and Discussion

7.1 Mass–Spring–Damper

The mass–spring–damper system successfully demonstrated the basic principles of state-space modeling. The system was represented using two states corresponding to displacement and velocity. The step and impulse responses of the state-space and transfer-function models overlapped, confirming that both representations describe the same system dynamics.

The individual-state analysis further demonstrated an important advantage of state-space modeling: the position and velocity states can be examined separately rather than treating only the final system output.

7.2 Eigenvalue Study

The eigenvalues of the system matrix were found to coincide with the poles of the corresponding system. Parameter variation demonstrated that changing mass, damping, and stiffness changes the location of the eigenvalues and therefore changes the dynamic characteristics of the system.

Increasing damping moved the system from oscillatory complex eigenvalues toward real negative eigenvalues, indicating suppression of oscillatory behavior. Increasing stiffness increased the imaginary component of the eigenvalues, indicating a higher oscillatory frequency. Increasing mass moved the eigenvalues toward the imaginary axis and reduced their oscillatory frequency.

7.3 Controllability

The mass–spring–damper system achieved full controllability because the controllability matrix had rank equal to the number of states. The deliberately constructed zero-input test case produced a rank-deficient controllability matrix, providing a simple demonstration of the importance of the input in controlling system states.

7.4 Aircraft State-Space Model

The methodology was subsequently applied to the third-order aircraft pitch plant. The aircraft state-space model contained three states and produced eigenvalues identical to the original transfer-function poles.

The controllability rank was three, confirming complete controllability, while the observability rank was also three, confirming complete observability.

The final transfer-function/state-space response comparison showed overlapping responses, providing verification that the state-space representation accurately captures the dynamics of the original aircraft model.

8 Aerospace Applications

State-space representation is particularly important in aerospace engineering because aircraft dynamics are naturally multi-variable and involve multiple coupled states.

An aircraft can simultaneously involve variables such as:

- position
- velocity
- angular velocity
- attitude
- aerodynamic states
- actuator states

Representing such systems through individual transfer functions can become cumbersome as the number of variables increases.

State-space modeling provides a systematic framework for representing these coupled dynamics using matrices.

For aircraft control, the concepts of controllability and observability are especially important. A controllable system can potentially be driven toward desired dynamic behavior through appropriate control inputs, while an observable system allows its internal states to be estimated from available measurements.

The aircraft pitch model developed in this project therefore provides a direct foundation for advanced control design.

9 Conclusion

This project investigated state-space representation using both a mass–spring–damper system and an aircraft pitch-control system. The mass–spring–damper model provided a simple platform for understanding state variables, system matrices, eigenvalues, state responses, and controllability.

The same methodology was then applied to the third-order aircraft pitch plant. The aircraft state-space model was successfully constructed and verified against the original transfer function. The eigenvalues of the system matrix matched the poles of the aircraft model, while rank analysis demonstrated complete controllability and observability.

The project therefore established a practical understanding of state-space modeling and demonstrated its application to an aerospace control system. Most importantly, it provides the mathematical foundation required for moving from classical control methods toward modern techniques based on state feedback and state estimation.

10 Future Work

Project 09 conclusively established the foundation for why the modern control methods are required achieve for this plant? The following projects will investigate modern control methods that overcome those limitations faced by classical control methods:

Project 10 — Pole Placement Control:

Full-state feedback will be investigated to directly assign the closed-loop poles of the aircraft pitch-control system to desired locations. The desired pole locations identified during the earlier classical-control studies will be revisited and used as targets for state-feedback design. This will allow the transient performance obtained through modern control to be compared with the results achieved using classical compensation.

Project 11 — LQR Optimal Control:

The Linear Quadratic Regulator will be introduced as an optimal state-feedback technique. Instead of selecting controller gains only to place poles at predetermined locations, LQR will determine feedback gains by minimizing a cost function that balances system performance against control effort. Its performance will be evaluated on the aircraft pitch-control system.

Project 12 — Kalman Filter / State Estimation:

State-space control generally requires knowledge of the system states. Since not every state can necessarily be measured directly in a practical aerospace system, state estimation will be investigated. A Kalman filter will be studied as a method for estimating unmeasured states from available sensor measurements while accounting for model and measurement uncertainty.

Projects 13–17 — Aerospace Control Applications:

The control-system tools developed throughout the mastery series will progressively be applied to more complex aerospace systems. Planned applications include UAV attitude control, rocket attitude control, satellite attitude determination and control, missile guidance and control, and finally an integrated flight-control-system capstone project.

11 Appendix

11.1 MATLAB Code

```
clc , clear all ; close all

%% Mass Spring Damper System
m = 1 ; b = 1 ; k = 1;

% ----- State Space Model -----
A = [0 1; -k/m -b/m]; % System Matrix
B = [0 ; 1]; % Input Matrix
C = [1 0]; % Output Matrix
D = [0]; % Feedback Matrix

sys_ss = ss(A,B,C,D)
poles = pole(sys_ss)
eigenValues = eig(A)

% ----- Transfer Function -----
% sys_tf = tf(sys_ss)
num = 1 ;
den = [m b k];
sys_tf = tf(num,den)

% ----- Verification -----
figure(1)
step(sys_ss,'r') ; hold on
step(sys_tf,'b--') ;
grid on
legend('State Space', 'Transfer Function');
title('Verification: Step Response Comparison');

figure(2)
impulse(sys_ss,'r') ; hold on
impulse(sys_tf,'b--') ;
grid on
legend('State Space', 'Transfer Function');
title('Verification: Impulse Response Comparison');
```

```

%% ----- Visualizing the States one-by-one -----

% There are two states:
%1. Position(x1)
%2. Velocity(x2/x1dot)

m = 1 ; b = 1 ; k = 1;
A = [0 1;-k/m -b/m];
B = [0 ; 1];
D = [0];

C_position = [1 0]; % Output is only x1(Position)
sys_position = ss(A,B,C_position,D);

C_velocity = [0 1]; % Output is only x2(Velocity)
sys_velocity = ss(A,B,C_velocity,D);

figure

step(sys_position,'r')
hold on
step(sys_velocity,'b--')

grid on
legend('Position x_1','Velocity x_2','Location','best')
title('State Responses')

%% ----- Parameter Study -----

% Project 01 : m,b,k ---> G(s) ---> poles ---> Response

% Project 09 : m,b,k ---> A ---> eigen values ---> State
Evaluation

B = [0 ; 1]; % Input Matrix
C = [1 0]; % Output Matrix
D = 0; % Feedback Matrix

%% Effect of mass

mVal = [1,2,5,10];
k = 1 ; b = 1;

figure
hold on ; grid on

```

```

for i = 1:length(mVal)
    m = mVal(i);
    A = [0 1; -k/m -b/m]; % System Matrix
    fprintf('eigen values of A when m = %d:\n',m)
    eigenValues = eig(A);
    disp(eigenValues)
    plot(real(eigenValues), imag(eigenValues), 'x', ...
        'MarkerSize', 10, 'LineWidth', 2)

```

```
end
```

```

xlabel('Real Axis')
ylabel('Imaginary Axis')
legend('m = 1', 'm = 2', 'm = 5', 'm = 10')
title('Eigenvalue Movement with Increasing Mass')

```

%% Effect of damper

```

bVal = [1,2,5,10];
k = 1 ; m = 1;

```

```

figure
hold on ; grid on

```

```

for i = 1:length(bVal)
    b = bVal(i);
    A = [0 1; -k/m -b/m]; % System Matrix
    fprintf('eigen values of A when b = %d:\n',b)
    eigenValues = eig(A);
    disp(eigenValues)
    plot(real(eigenValues), imag(eigenValues), 'x', ...
        'MarkerSize', 10, 'LineWidth', 2)

```

```
end
```

```

xlabel('Real Axis')
ylabel('Imaginary Axis')
legend('b = 1', 'b = 2', 'b = 5', 'b = 10')
title('Eigenvalue Movement with Increasing Damping')

```

```

                                %% Effect of friction

kVal = [1,2,5,10];
b = 1 ; m = 1;

figure
hold on ; grid on

for i = 1:length(kVal)
    k = kVal(i);
    A = [0 1;-k/m -b/m]; % System Matrix
    fprintf('eigen values of A when k = %d:\n',k)
    eigenValues = eig(A);
    disp(eigenValues)
    plot(real(eigenValues), imag(eigenValues), 'x', ...
        'MarkerSize', 10, 'LineWidth', 2)

end
xlabel('Real Axis')
ylabel('Imaginary Axis')
legend('k = 1','k = 2','k = 5','k = 10')
title('Eigenvalue Movement with Increasing Friction')


%% ----- Controllability -----
% To see whether the input (force) affects both the states of
% system or not?
% if Rank(ControlledMatrix) = number of states

m = 1 ; b = 1 ; k = 1;

A = [0 1;-k/m -b/m]
B = [0 ; 1]
n = 2

Co = ctrb(A,B)
Rank = rank(Co)

if Rank == n
    disp('The system is completely controllable')
else
    disp('The system is not controllable')
end

```



```

% Test Case : Uncontrollable System

A_test = [0 1;-k/m -b/m]
B_test = [0 ; 0] % No input is applied

Co_test = ctrb(A_test,B_test)
Rank_test = rank(Co_test)

if Rank_test == n
    disp('The system is compleety controllable')
else
    disp('The system is not controllable')
end

%% --- State Space Analyis of Aircract Pitch Control System

% system of state-space
A = [0 1 0 ; 0 0 1 ; -0.179 -0.987 -1.935]; % System Matrix
B = [0;0;1]; % Input Matrix
C = [1.282 -1.282 0]; % Output Matrix
D = 0; % Feedback through
Matrix

disp('----- STATE SPACE RESPRESENTAION -----')
sys_ss = ss(A,B,C,D)

disp('----- TRANSFER FUNCTION RESPRESENTAION ----')
sys_tf = tf(sys_ss)

% Poles and Eigen Values

poles = pole(sys_ss);
disp('Poles of the system are:')
disp(poles)

eigenValues = eig(A);
disp('Eigen values of the system are:')
disp(eigenValues)

```

```

%% ----- Controllability -----
A = [0 1 0 ; 0 0 1 ; -0.179 -0.987 -1.935];    % System Matrix
B = [0;0;1];                                    % Input Matrix
C = [1.282 -1.282 0];                          % Output Matrix
D = 0;                                           % Feedback through
Matrix

Co = ctrb(A,B);
disp('Controllable Matrix:')
disp(Co)

R = rank(Co);
disp('Rank:')
disp(R)

%% ----- Observability -----

A = [0 1 0 ; 0 0 1 ; -0.179 -0.987 -1.935];    % System Matrix
B = [0;0;1];                                    % Input Matrix
C = [1.282 -1.282 0];                          % Output Matrix
D = 0;                                           % Feedback through
Matrix

Ob = obsv(A,C);
disp('Observability Matrix:')
disp(Ob)

R = rank(Ob);
disp('Rank:')
disp(R)

%% ----- SS & Tf -----
t = 0:0.01:30;
[y_tf,t_tf] = step(sys_tf,t);
[y_ss,t_ss] = step(sys_ss,t);
figure
plot(t_tf,y_tf,'LineWidth',1.5)
hold on
plot(t_ss,y_ss,'--','LineWidth',1.5)
grid on
xlabel('Time (s)')
ylabel('Pitch Angle')
title('Aircraft Pitch Response: Transfer Function vs State-
Space')
legend('Transfer Function','State-Space')

```